

Sistemas Multiagente

Tema 2: Herramientas y Plataformas de agentes

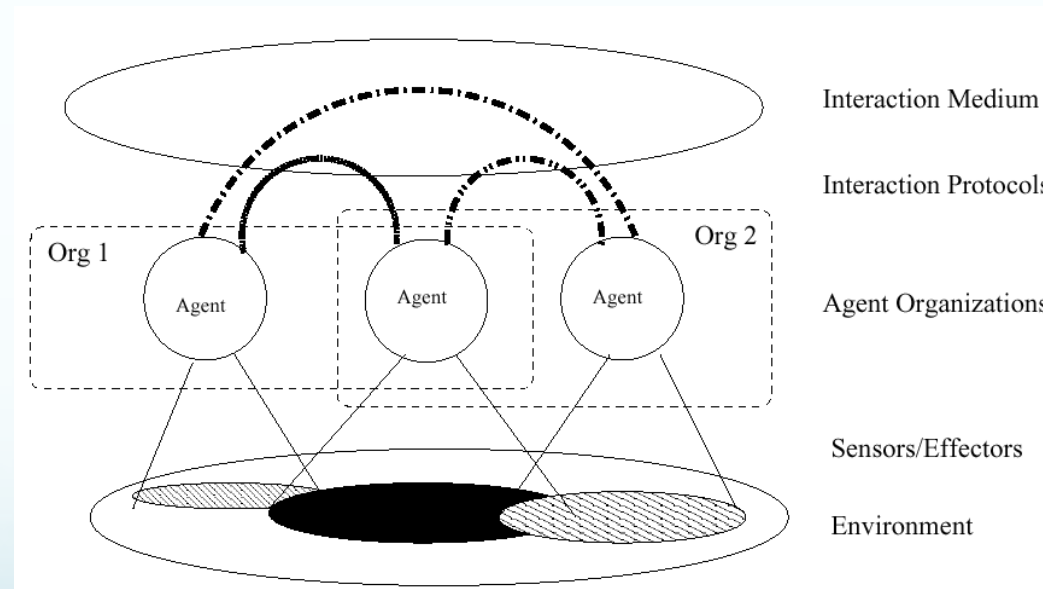
Authors: Vicent Botti, Vicente Julián, Javier Palanca

Conceptos. Introducción

- Problema a resolver:
 - Modelar sistemas reales complejos y con características claramente distribuidas
- Visión de un sistema como una organización computacional consistente de varios “roles” interactuando.
- Diseño (Métodos) → Implementación (Lenguajes) → Implantación (Middleware)

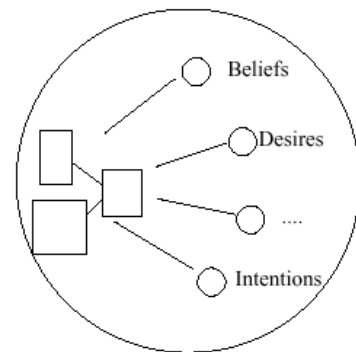
Conceptos. Introducción

- Identificar:
 - los diferentes subsistemas que forman parte del sistema global
 - las posibles interacciones y dependencias entre ellos



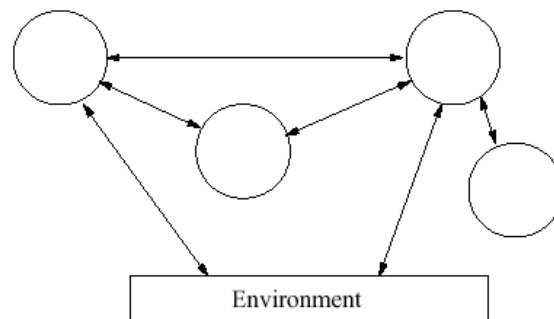
Conceptos. Introducción

- A tener en cuenta:
 - punto de vista interno → un agente
 - punto de vista externo → varios agentes



The Intra-Agent Viewpoint

(agent's own purpose, internal structure, technology)



The Inter-Agent Viewpoint

(interaction with the environment and with other agents)

La necesidad de utilizar una metodología

Desarrollar SMA es complejo

Hay soluciones tecnológicas:

- Arquitecturas: subsunción, BDI, capas, ...
- Entornos o Lenguajes de desarrollo: AgentBuilder, JADEL...
- Plataformas de ejecución: JADE, Jack, Jason... SPADE

Pero hace falta responder a varias cuestiones:

- Qué problema tengo que resolver
- Desarrollar muchos agentes sencillos o pocos agentes de elevada complejidad
- Explicar a otros cómo es el SMA
- Trabajar más de una persona: Coordinar el trabajo
- Valorar la calidad del desarrollo
- Qué proceso seguir
- ...

Plataformas y Middleware

¿Cómo ejecutamos un sistema multiagente?

Plataformas de Agentes

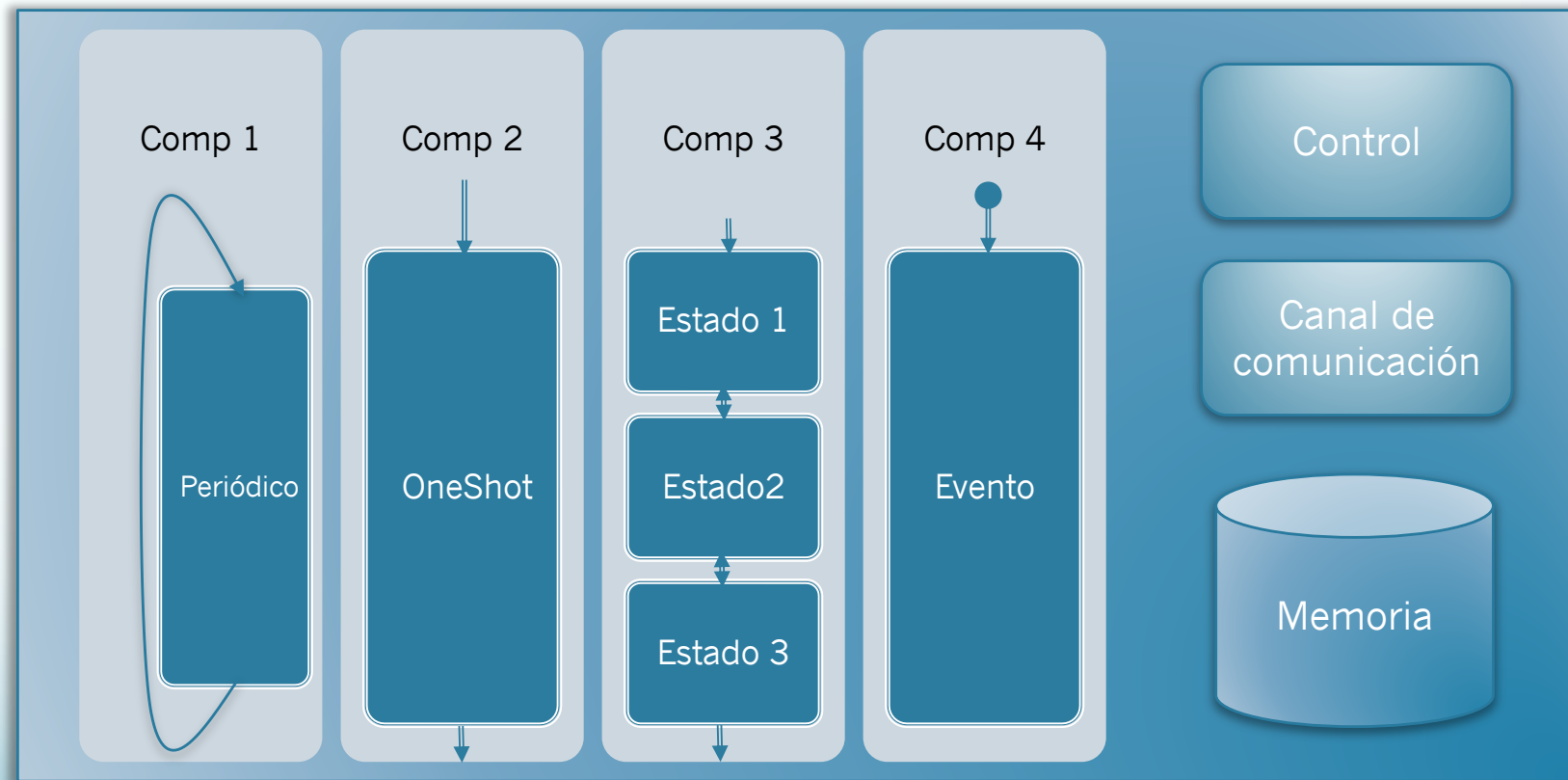
- ¿Qué nos debería ofrecer una plataforma de agentes?
- ¿Qué plataformas hay existentes?

Plataformas

- Funcionalidades:
 - Proporciona un **alto nivel de abstracción** para el desarrollo de determinadas aplicaciones
 - Conjunto de **herramientas** que facilitan el desarrollo, fundamentalmente la **comunicación**
 - **API** adaptada al dominio de las aplicaciones
 - Facilitan la **interoperabilidad**

Plataformas

- Modelo Computacional: Basado en comportamientos



Plataformas existentes

- Muchos *intentos* de realizar la plataforma definitiva
- Focalización únicamente en unos pocos aspectos
 - Mundo de los agentes demasiado amplio para hacer una plataforma “*definitiva*”
 - Cada plataforma enfocada a los temas de investigación de la institución que la desarrolla
- Pocos estándares ampliamente aceptados
 - FIPA (<http://www.fipa.org>)
- Posibilidad de crear sistemas “multiagente” utilizando tecnologías no únicamente de agentes

(algunas) Plataformas

- **Jade** (BELLIFEMINE, F., Caire, G., Poggi, A. & Rimassa, G. (2003). JADE: A white Paper. *EXP in search of innovation*, 3(3), 6–19.)
- **JadeX** (BRAUBACH, L. & Pokahr, A. (2013). The Jadex Project: Simulation. *Multiagent Systems and Applications*, 45, 107–128)
- **JACK** (<http://aosgrp.com/products/jack/>), **CoJACK** (<http://aosgrp.com/products/cojack/>) y **C-BDI** (<http://aosgrp.com/products/c-bdi/>)
- **Jason** (BORDINI, R., Hübner, J. & Wooldridge, M. (2007). *Programming Multi-Agent Systems in AgentSpeak using Jason*. Wiley Series in Agent Technology)
- **Magentix2** (magentix2.gti-ia.upv.es/)
- **PADE** (<https://github.com/grei-ufc/pade>)
- **Spade** (<https://pypi.org/project/SPADE/>)

Para simulación: **NetLogo**, **Repast**, **AnyLogic**, **GAMA**, **MATSim**...

(algunas) Plataformas existentes

Pal, C. V., Leon, F., Paprzycki, M., & Ganzha, M. (2020). A Review of Platforms for the Development of Agent Systems. *arXiv preprint arXiv:2007.08961*

A Review of Platforms for the Development of Agent Systems

Constantin-Valentin Pal¹, Florin Leon¹, Marcin Paprzycki², Maria Ganzha²

¹ *Faculty of Automatic Control and Computer Engineering
“Gheorghe Asachi” Technical University of Iași, Romania*

² *Systems Research Institute
Polish Academy of Sciences*

valentin.pal@tuiasi.ro, florin.leon@tuiasi.ro,
paprzyck@ibspan.waw.pl, maria.ganzha@ibspan.waw.pl

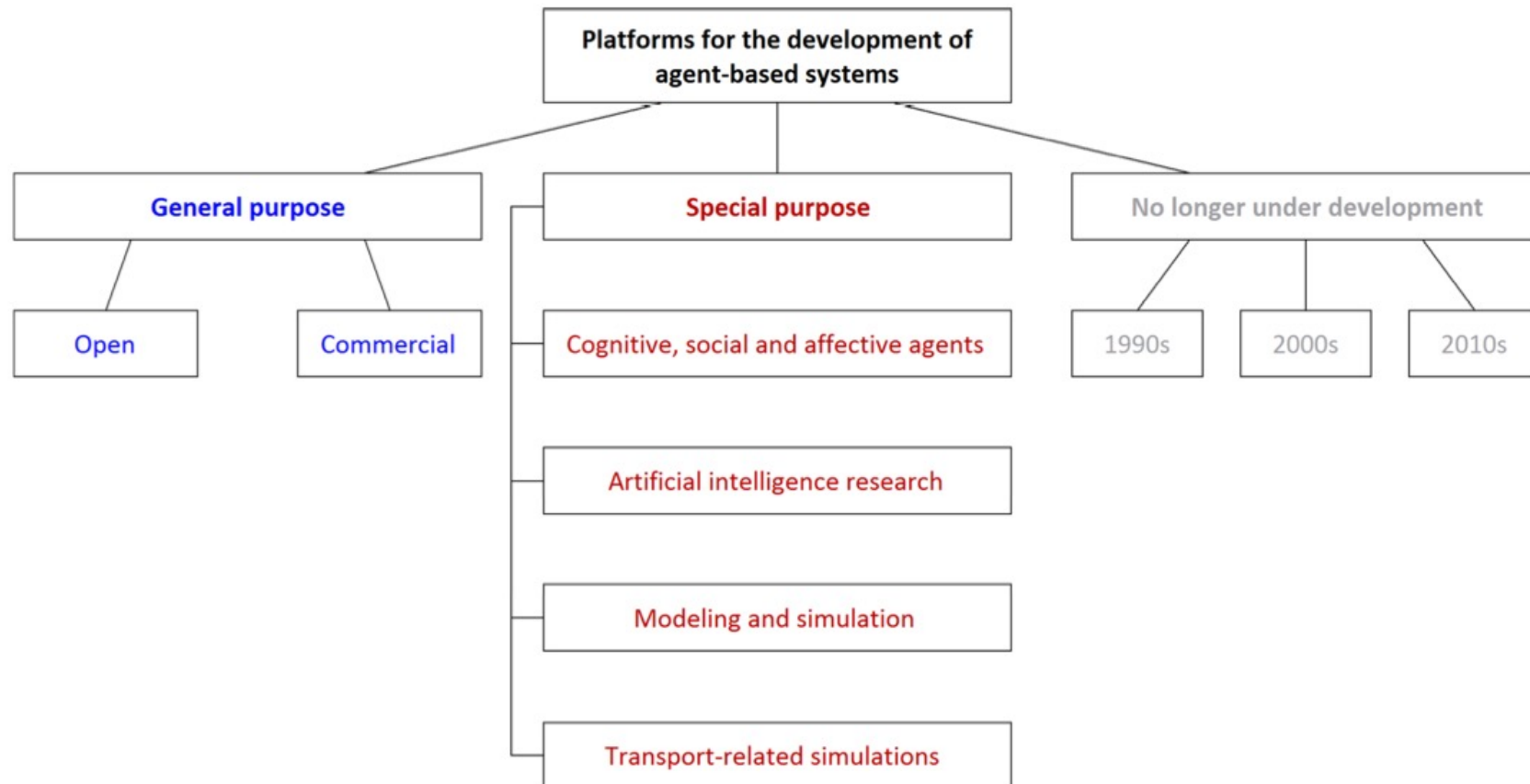
Abstract

Agent-based computing is an active field of research with the goal of building autonomous software of hardware entities. This task is often facilitated by the use of dedicated, specialized frameworks. For almost thirty years, many such agent platforms have been developed. Meanwhile, some of them have been abandoned, others continue their development and new platforms are released. This paper presents a up-to-date review of the existing agent platforms and also a historical perspective of this domain. It aims to serve as a reference point for people interested in developing agent systems. This work details the main characteristics of the included agent platforms, together with links to specific projects where they have been used. It distinguishes between the active platforms and those no longer under development or with unclear status. It also classifies the agent platforms as general purpose ones, free or commercial, and specialized ones, which can be used for particular types of applications.

Keywords: agent systems, agent-based systems, multiagent systems, agent platforms, modeling, simulation, swarm intelligence, artificial intelligence

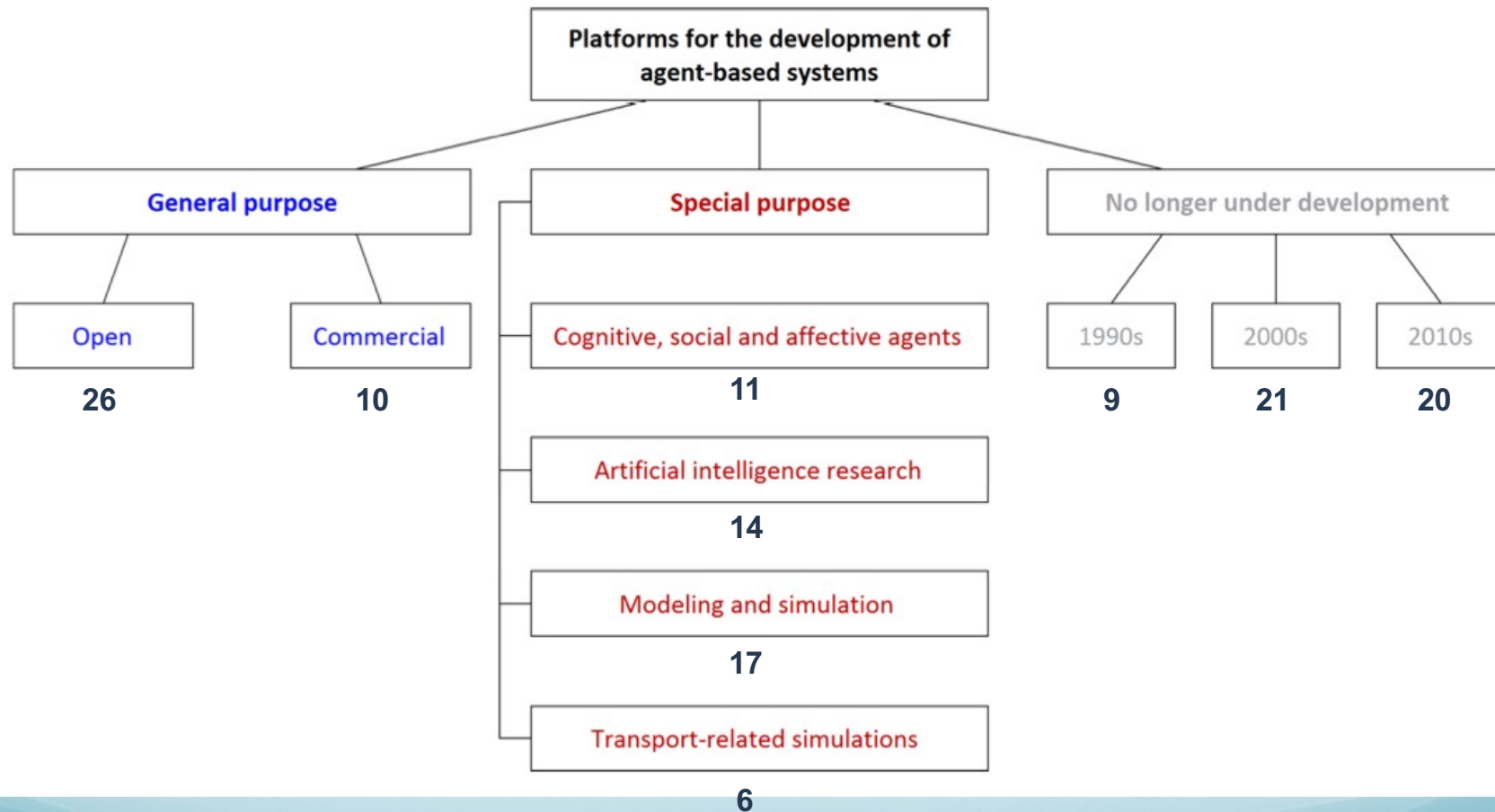
(algunas) Plataformas existentes

Pal, C. V., Leon, F., Paprzycki, M., & Ganzha, M. (2020). A Review of Platforms for the Development of Agent Systems. *arXiv preprint arXiv:2007.08961*



(algunas) Plataformas existentes

Pal, C. V., Leon, F., Paprzycki, M., & Ganzha, M. (2020). A Review of Platforms for the Development of Agent Systems. *arXiv preprint arXiv:2007.08961*



¿Cuál era la más usada?



- JADE (Java Agent Development Framework) : Marco de trabajo para el desarrollo de SMA y aplicaciones acorde con el estándar FIPA:
 - Plataforma para agentes (FIPA-compliant).
 - Paquete para el desarrollo de agentes en Java.
- <http://jade.tilab.com> (versión 4.6.0 del 2022)

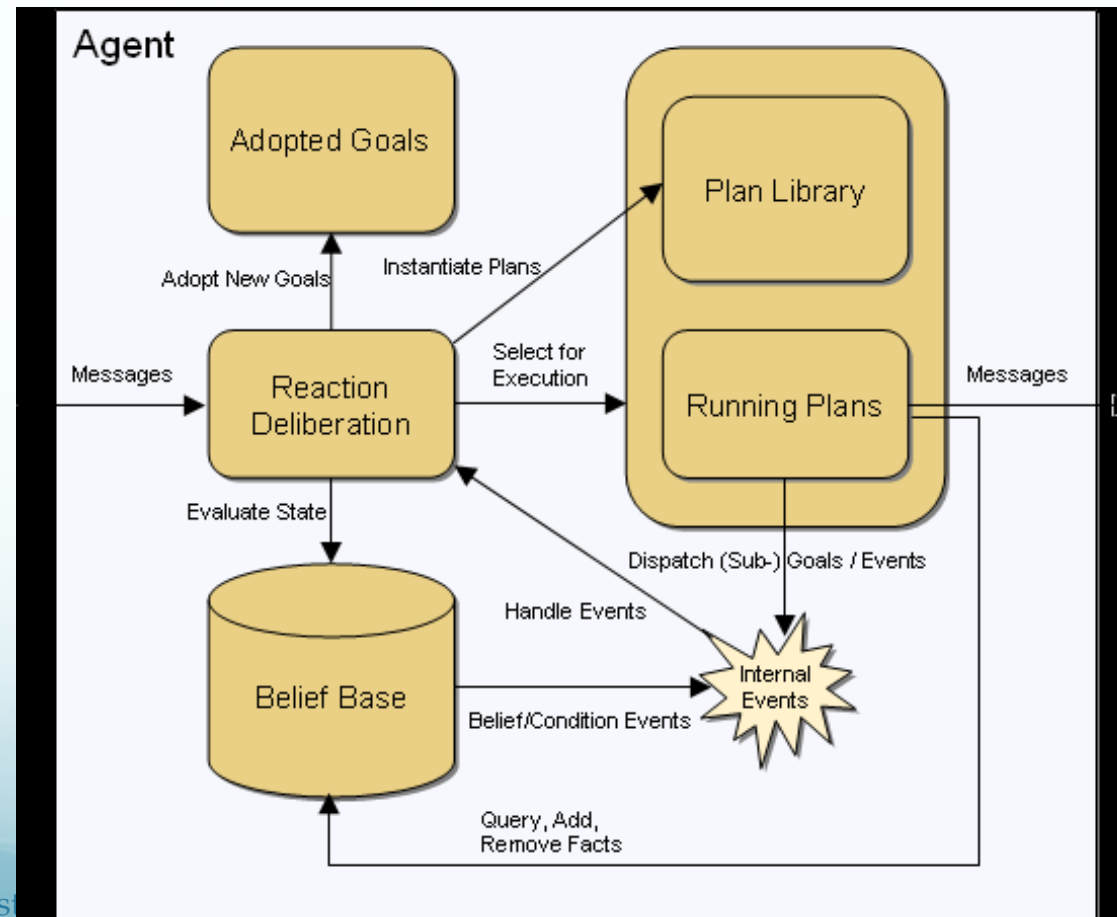
JADE: Principales características

- Plataforma distribuida de Agentes.
- Interfaz grafica.
- Herramientas de depuración.
- Movilidad de los agentes intra-plataforma.
- Soporte para la ejecución de múltiples actividades de los agentes (Behaviours).
- Plataforma FIPA-Compliant.
 - AMS, DF y ACC
- Librería de protocolos de interacción FIPA.
- Registro y desregistro automático de agentes con el AMS.
- Permite usar Ontologías definidas y lenguajes de Contenido.

Plataformas existentes: JADEX

- JADE + Diseño BDI de los agentes:

- <https://www.activecomponents.org/>
- <https://github.com/actoron/jadex>



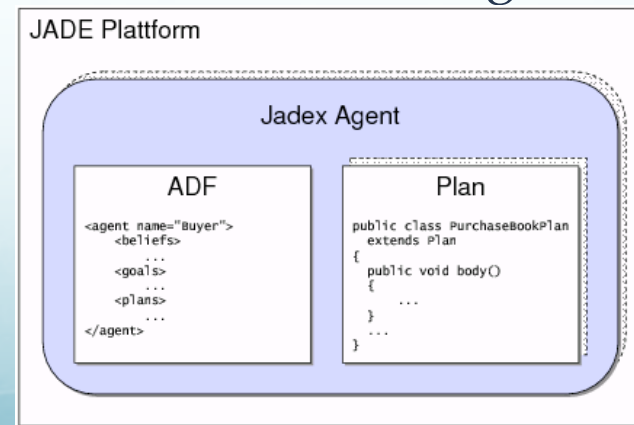
JADEx

Extensión de Jade

Ultima versión: 15/6/2021 Jadex 4.0.241 Release

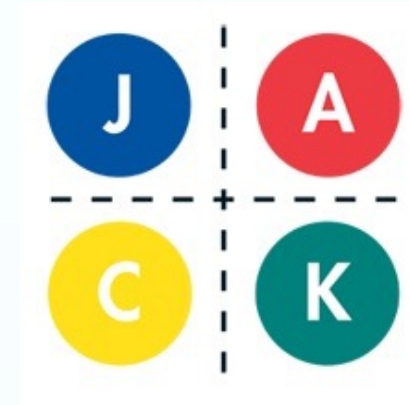
Los agentes Jadex:

- Se definen con dos elementos: una descripción en xml y clases java que representan sus planes.
- La implementación en java de cada plan accede y modifica las creencias, satisface / abandona objetivos y adopta nuevos objetivos.
- En el fichero xml incluimos información sobre: creencias, deseos (goals), planes, mensajes y estados iniciales del agente.



Plataformas existentes: Jack

- Jack está desarrollado por la empresa **AOS Group**
- No es de libre distribución
- Desarrollada en Java
- Basada en BDI
- <http://aosgrp.com/products/jack/>
 - version 5.6
- Recientemente hay nuevas versiones
 - CoJACK: para modelar el comportamiento humano. Se utiliza en sistemas de simulación para apoyar a actores virtuales.
 - C-BDI: apropiada para sistemas embebidos. Centrado en sistemas aeroespaciales y de defensa autónomos y semiautónomos en misión crítica. Diseñado para su uso en dominios en tiempo real
 - Jack Teams
 - Jack SIM



Plataformas existentes: Jason

- Libre distribución (GNU LGPL)
- Desarrollada en Java
- Basada en BDI: Uso de AgentSpeak
- <http://jason.sourceforge.net/wp/>
- Última versión 3.1 (23/03/2022)
- Principales características
 - Jason es un intérprete de una versión extendida de AgentSpeak.
 - Implementa la semántica operacional de ese lenguaje.
 - Provee una plataforma para el desarrollo de sistemas multi-agente



Plataformas existentes: Janus (SARL)

- Plataforma de código abierto totalmente implementada en Java
- Maneja de forma nativa el concepto de agentes recursivos y holones.
- Soporta el Lenguaje de Programación Orientado a Agentes SARL.
- Ofrece: concurrencia, distribución, interacción, descentralización, reactividad, autonomía y reconfiguración dinámica
- SARL facilita la ejecución de agentes y la comunicación directa.
- Se proporciona una extensión para soportar los entornos simulados.
- <http://www.sarl.io> Última versión V3 0.12.0 (27/06/2021)



Plataformas existentes:

Entornos de simulación

- Las veremos en el último tema
- Enfocadas a sistemas a gran escala con agentes ligeros
- Ejemplos:
 - Netlogo - <http://ccl.northwestern.edu/netlogo/>
 - Anylogic - <https://www.anylogic.com>
 - MATSim - <https://www.matsim.org>
 - Gama - <https://gama-platform.github.io>
 - Repast - <https://repast.github.io>
 - JackSim - <http://aosgrp.com/products/jack-simtm/>
 - MESA - <https://mesa.readthedocs.io/en/stable/>

Otras opciones ... SPADE

- Plataforma desarrollada en Python
- Y que vamos a ver en detalle ...

SPODE

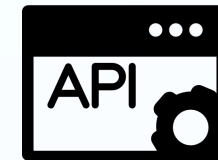
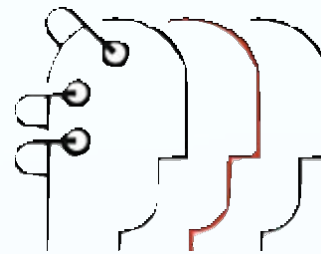
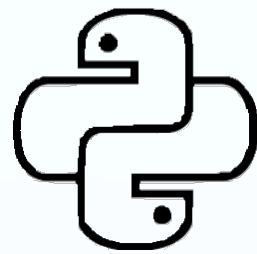
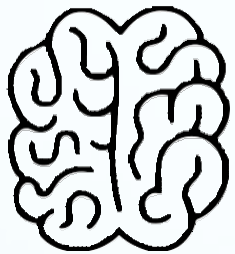
SMART PYTHON AGENT
DEVELOPMENT ENVIRONMENT

3.X



SPADE

Smart Python Agent Dev Env

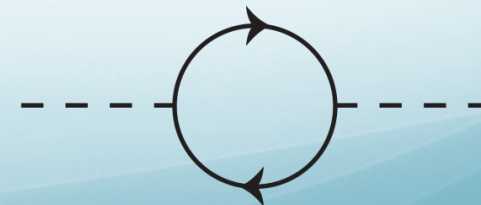


Basado en:

Python ≥ 3.6

JABBER / XMPP

AsyncIO





¿Qué es Jabber / XMPP?

- ★ Jabber es un protocolo abierto basado en XML para mensajería instantánea y notificación de presencia.
- ★ Proyecto iniciado en 1998 por Jeremie Miller.
 - ★ Primera release en 2000.
- ★ Estandarizado por la IETF y el W3C para la mensajería instantánea a través de Internet.
- ★ Ahora se llama XMPP por **eXtensible Messaging and Presence Protocol**



¿Cómo es Jabber?

Abierto, público y gratuito

Estándar

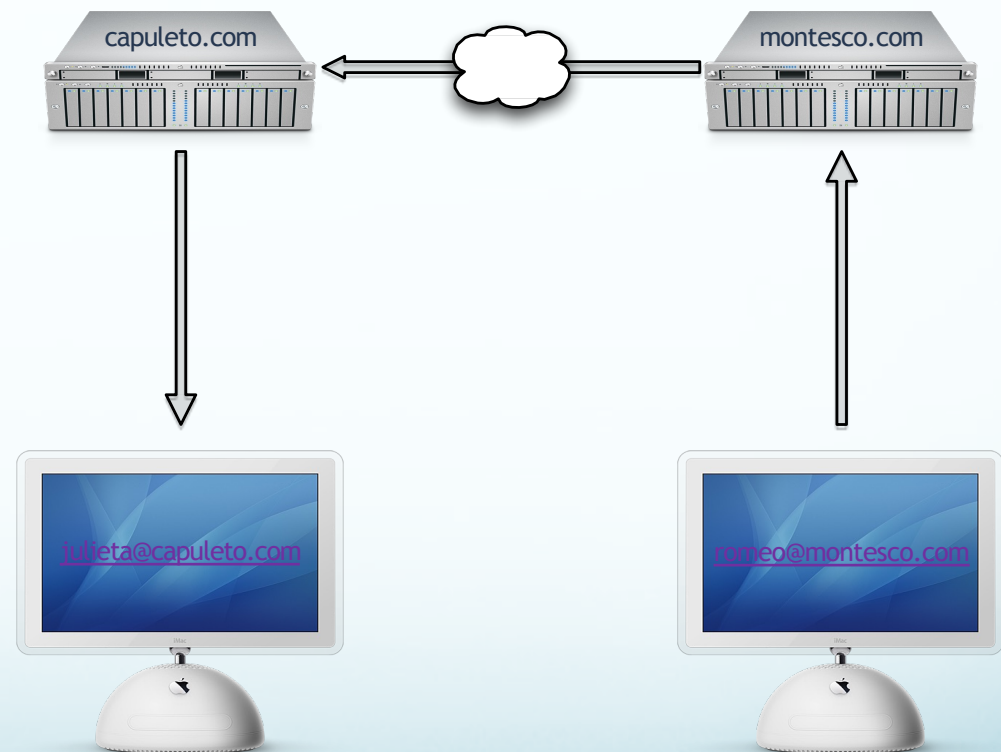
Testado

Descentralizado

Seguro

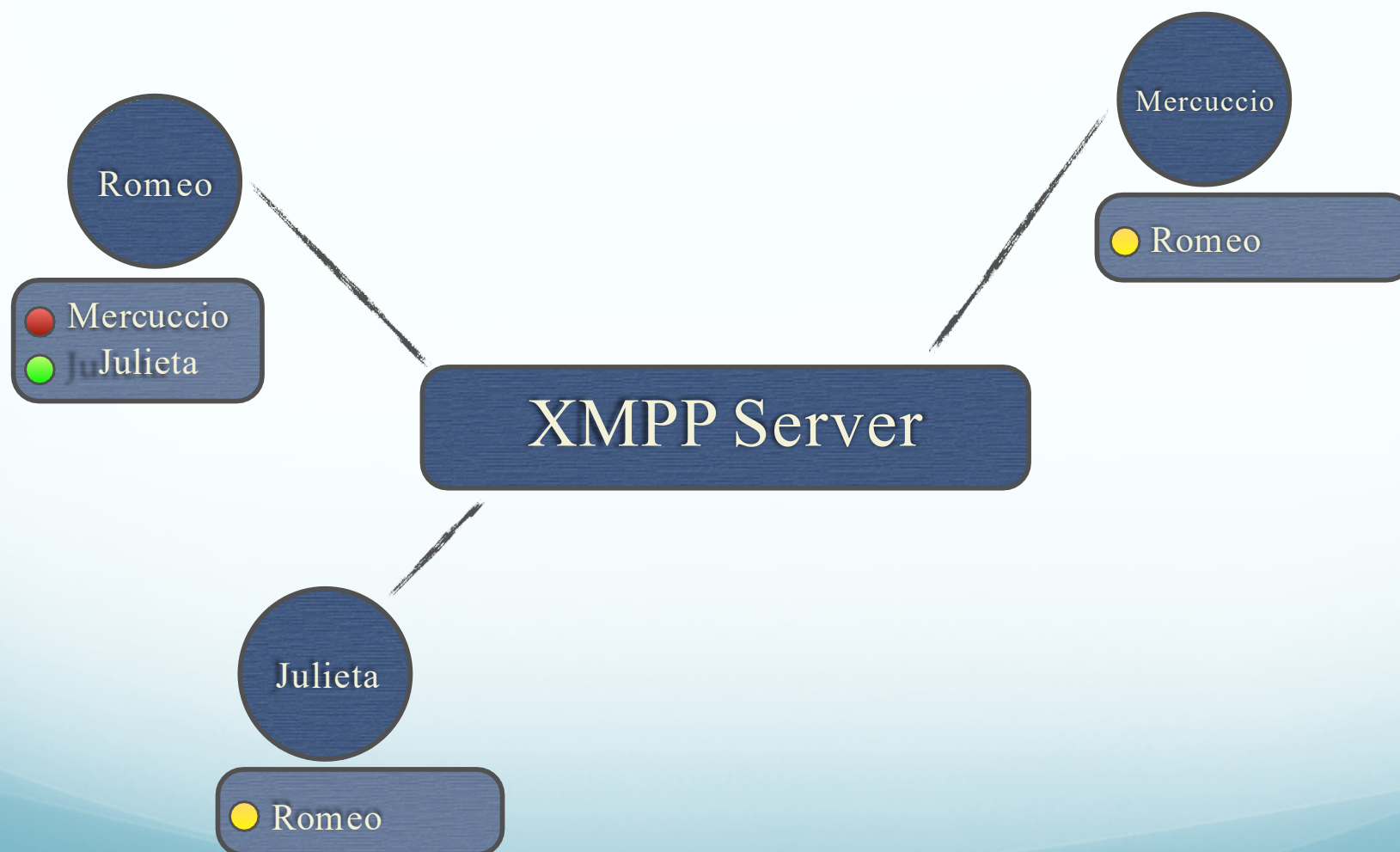
Extensible

Flexible



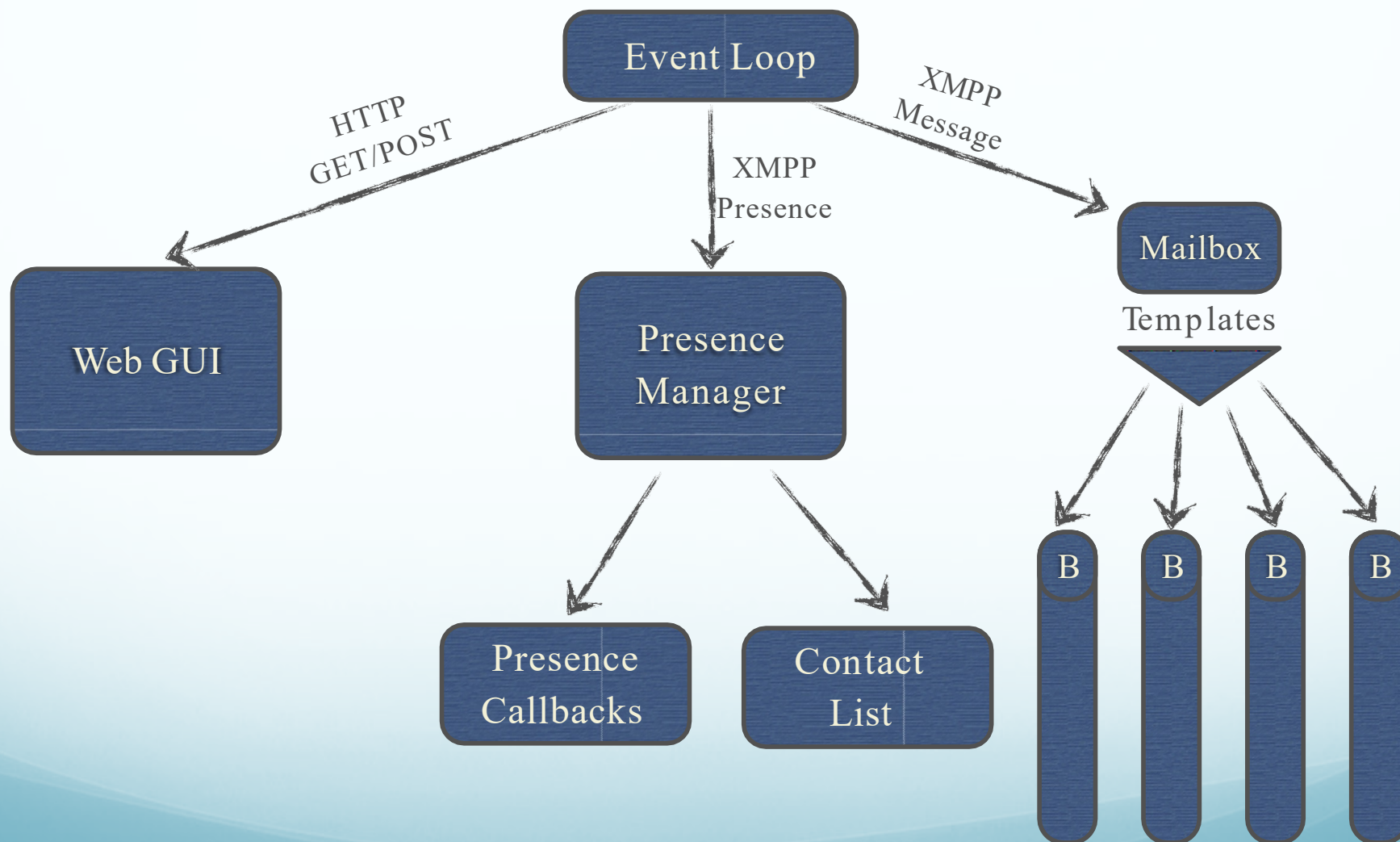


Notificación de Presencia





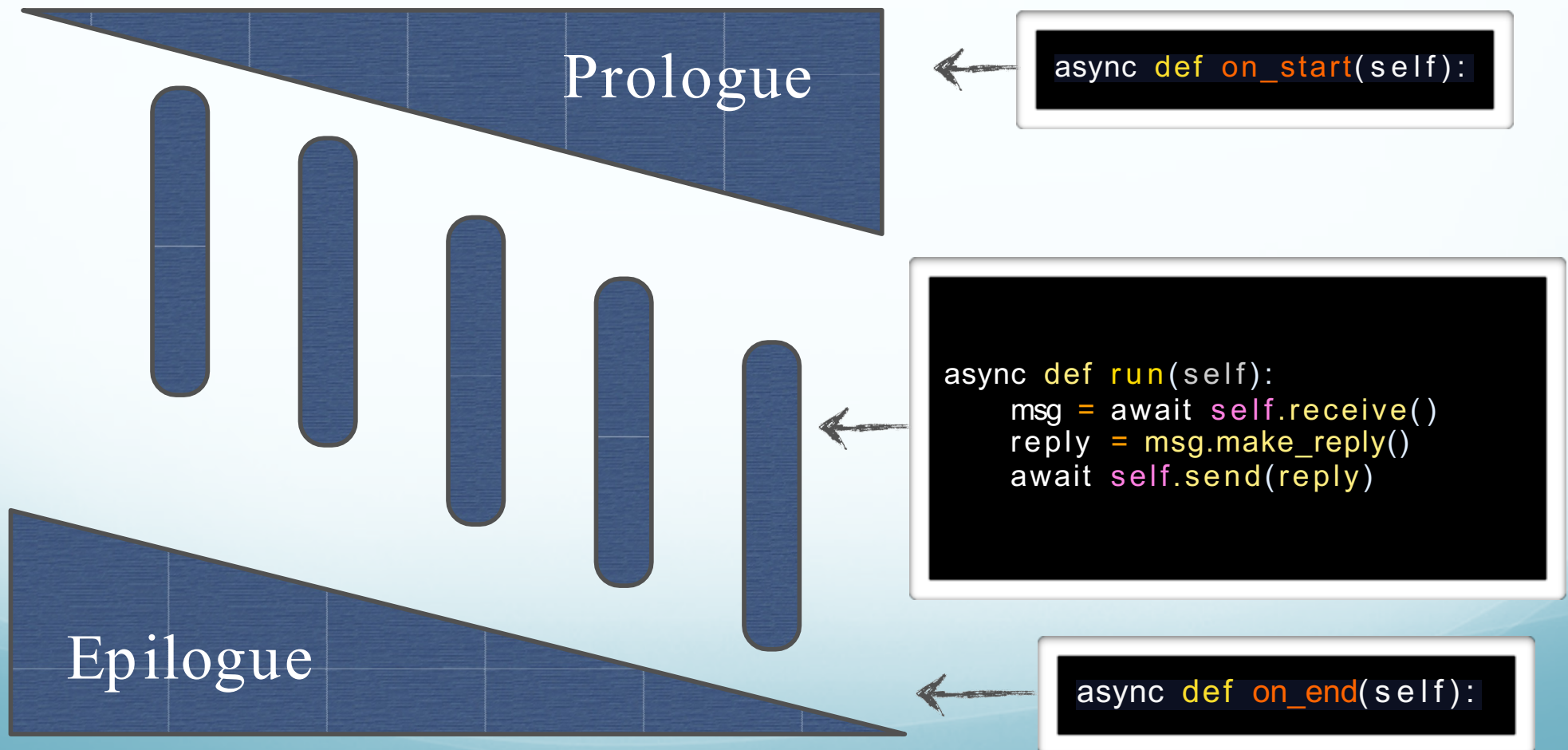
Arquitectura de Agente





Ciclo de vida

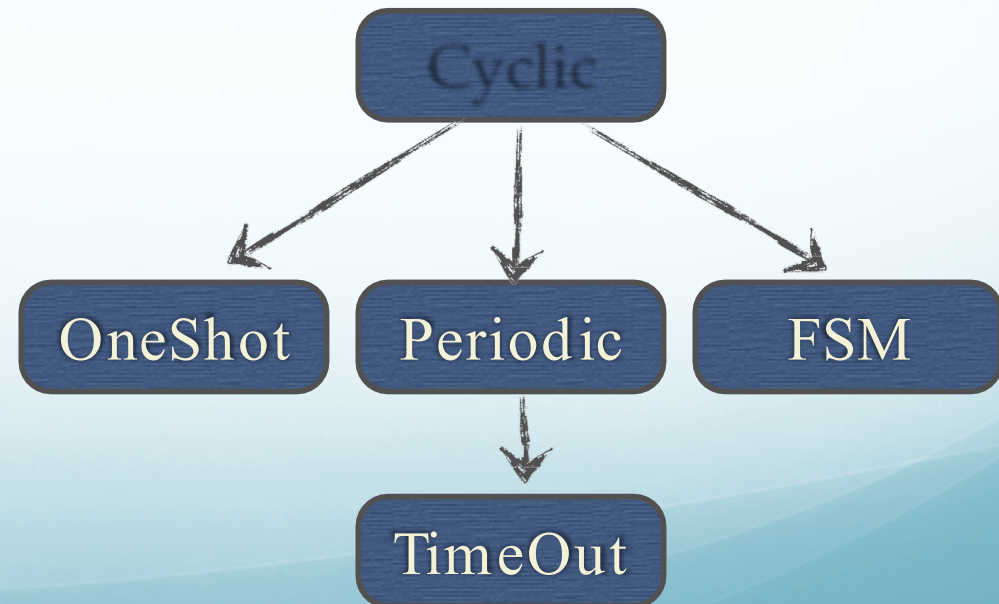
Basado en Async Behaviors





Comportamientos

- ⊙ Comportamientos asíncronos
- ⊙ Cada comportamiento tiene su propio buzón
- ⊙ Plantillas de mensajes
- ⊙ Protocolos de interacción con Templates



Interfaz de usuario



SPADE



your_jid
Online

Dashboard

Dashboard

Behaviours

CyclicBehaviour/DummyBehav

Template: <template to="None" from="agent0@fake_server" thread="..."

Kill

PeriodicBehaviour/DummyPeriodBehav

Template: <template to="None" from="agent1@fake_server" thread="..."

Kill

TimeoutBehaviour/DummyTimeoutBehav

Template: <template to="None" from="agent2@fake_server" thread="..."

Kill

FSMBehaviour/DummyFSMBehav

Template: <template to="None" from="agent3@fake_server" thread="None" ...

Contacts



agent0@fake_se...
Online



agent1@fake_se...
Away



agent2@fake_se...
DND



agent4@fake_se...
Online



agent3@fake_se...
Offline



agent5@fake_se...
Offline

Copyright © 2018 SPADE.

Version 3.0.0

Interfaz de usuario



SPADE

 **your_jid**
Online

Dashboard

FSMBehaviour/DummyFSMBehav

Home > Dashboard

4

Mailbox



True

Is killed?



S 1

Current State



Template

```
<template to="None" from="agent3@fake_server" thread="None" metadata=[]></template>
```



0

Exit Code



Chat

8 — x

your_jid 11 minutes ago

This is my answer.



agent3 11 minutes ago

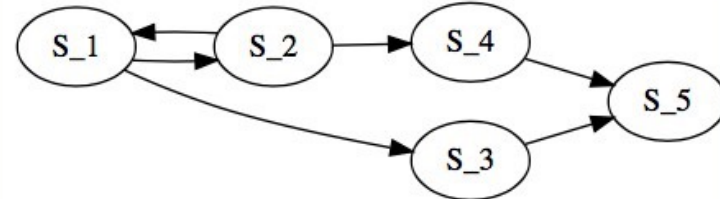
 Hello from agent3!
This is a long message.

your_jid 11 minutes ago

This is my answer.



Finite State Machine



Interfaz de usuario



SPADE



your_jid

Online

Dashboard



your_jid

agent0@fake_server

Home > Dashboard

agent0@fake_server



STATUS

ONLINE

ACTION

Unsubscribe

12
MESSAGES

Chat

12 — ×



Hello from agent0! This is a long message.

your_jid

2 minutes ago

This is my answer.



agent0

2 minutes ago



Hello from agent0! This is a long message.

your_jid

2 minutes ago

This is my answer.



Type Message ...

Send

Instalación

<https://spade-mas.readthedocs.io>

- ⊙ Se recomienda **Virtualenvs** (conda, Pipenv, ...)
- ⊙ Usar **Python** ≥ 3.6 (< 3.10)
- ⊙ Necesario instalar un servidor **XMPP** o usar uno público

Prosody en Linux

Openfire en Windows

- ⊙ Activar **in-band register** en el servidor XMPP

En Prosody es: **allow_registration=true** en el fichero prosody.cfg.lua

```
$ pip install spade # if using conda
$ pipenv install spade # if using pipenv
```

Instalación

En Linux:

```
# Create virtual environment with python = 3.9
```

```
# Conda or pipenv
```

```
$ conda create -n spade python=3.9
```

```
$ conda activate spade
```

```
# Install spade
```

```
(spade)$ pip install spade
```


Instalación

En Linux (Polilabs):

```
$ source ruta_anaconda
```

```
$ conda init
```

Abrir otro terminal

```
# Create virtual environment with python = 3.9
```

```
# Conda or pipenv
```

```
$ conda create -n spade python=3.9
```

```
$ conda activate spade
```

```
# Install spade
```

```
(spade)$ pip install spade
```

Instalación

Como servidor XMPP usaremos (solo en la red de la UPV) :

gtirouter.dsic.upv.es

CUIDADO!!! Es un servidor compartido

Cada agente debe tener un nombre diferente

Si se quiere instalar un servidor propio:

- **Prosody en Linux**
- **Openfire en Windows**

Mi primer agente

<https://spade-mas.readthedocs.io>

```
from spade import agent

class DummyAgent(agent.Agent):
    async def setup(self):
        print("Hello World! I'm agent {}".format(str(self.jid)))

dummy = DummyAgent("your_jid@your_xmpp_server", "your_password")
dummy.start()

dummy.stop()
```

```
$ python dummyagent.py
Hello World! I'm agent your_jid@your_xmpp_server
$
```

Mi primer comportamiento

```
import time
from spade.agent import Agent
from spade.behaviour import CyclicBehaviour

class DummyAgent(Agent):
    async def setup(self):
        print("Agent starting . . .")
        b = self.MyBehav()
        self.add_behaviour(b)

    class MyBehav(CyclicBehaviour):
        async def on_start(self):
            print("Starting behaviour . . .")
            self.counter = 0

        async def run(self):
            print("Counter: {}".format(self.counter))
            self.counter += 1
            await asyncio.sleep(1)

if __name__ == "__main__":
    dummy = DummyAgent("your_jid@your_xmpp_server", "your_password")
    dummy.start()

    # Wait until user interrupts with ctrl+C
    while True:
        try:
            time.sleep(1)
        except KeyboardInterrupt:
            break
    dummy.stop()
```

```
$ python dummyagent.py
Agent starting...
Starting behaviour...
Counter: 0
Counter: 1
Counter: 2
Counter: 3
Counter: 4
Counter: 5
Counter: 6
Counter: 7
```

Sender & Receiver

```
from spade.agent import Agent
from spade.behaviour import OneShotBehaviour
from spade.message import Message

class SenderAgent(Agent):
    class InformBehav(OneShotBehaviour):
        async def run(self):
            print("InformBehav running")
            msg = Message(to="receiver@xmpp_server")
            msg.set_metadata("performative", "inform")
            msg.body = "Hello World"

            await self.send(msg)
            print("Message sent!")

            # stop agent from behaviour
            await self.agent.stop()

    async def setup(self):
        print("SenderAgent started")
        self.b = self.InformBehav()
        self.add_behaviour(self.b)
```

```
from spade.agent import Agent
from spade.behaviour import OneShotBehaviour
from spade.template import Template

class ReceiverAgent(Agent):
    class RecvBehav(OneShotBehaviour):
        async def run(self):
            print("RecvBehav running")
            msg = await self.receive(timeout=10)
            if msg:
                print("Message received {}".format(msg.body))
            else:
                print("No message after 10 seconds")

            await self.agent.stop()

    async def setup(self):
        print("ReceiverAgent started")
        b = self.RecvBehav()
        template = Template()
        template.set_metadata("performative", "inform")
        self.add_behaviour(b, template)
```

```
if __name__ == "__main__":
    receiveragent = ReceiverAgent("receiver@xmpp_server", "receiver_password")
    receiveragent.start()
    time.sleep(2) # wait for receiver agent to be prepared
    senderagent = SenderAgent("sender@xmpp_server", "sender_password")
    senderagent.start()

    while receiveragent.is_alive():
        try:
            time.sleep(1)
        except KeyboardInterrupt:
            senderagent.stop()
            receiveragent.stop()
            break
```



SPADE

- **Free Software** project (LGPL)
- Abierto a colaboraciones (**PULLREQUEST!**)
- Contacto:
 - **web:** <http://github.com/javipalanca/spade>
 - **doc:** <https://spade-mas.readthedocs.io/>

Conclusiones

- La tecnología de middleware (plataforma) se ha convertido en algo necesario para desarrollar sistemas multiagente
- El mundo de los sistemas multiagente es muy amplio y no existen plataformas “universales”, foco en distintos aspectos:
 - Diseño interno del agente
 - Comunicaciones
 - Normas
 - Organizaciones
 - Negociación
 - Argumentación
 - Confianza y reputación
 - ...

Conclusiones

- Elección de plataforma
 - ¿Existe alguna que se adapte? El dominio importa
 - IOT
 - Cyber Physical Systems
 - Industria 4.0
 - Nivel de abstracción muy alto
 - Cada plataforma implementa los sistemas multiagente de una manera particular
 - Utilización de muchas tecnologías
 - Puesta en funcionamiento